

Machine-Learning-Based Cell Counting on a 3D-Printed Motorized Microscope Platform

Addison Drobný

Department of Computer Science
University of California, Irvine
Irvine, USA
addrobn@uci.edu

Dylan Le

Department of Computer Science
University of California, Irvine
Irvine, USA
dble2@uci.edu

Kirby Burke

Department of Electrical
Engineering and Computer Science
University of California, Irvine
Irvine, USA
kirbylb@uci.edu

Mina Li

Department of Computer Science
University of California, Irvine
Irvine, USA
minasl@uci.edu

Roberto Lucatero

Department of Computer Science
University of California, Irvine
Irvine, CA
lucatero@uci.edu

Abstract—This project presents the development of an accessible and low-cost solution for the automation of capturing accurate insights about the quantity and movement of microscopic elements for students, educators, and researchers via an open-source app, driven by machine-learning insights, that works with the OpenFlexure microscope and software ecosystem.

Keywords—3D printing, cell detection, digital imaging systems, embedded systems, image processing, machine learning, machine vision, microscopy, open-source platform, Raspberry Pi

I. BACKGROUND

Cell counting, a fundamental task in biological research, biomedical education, and clinical diagnostics, allows researchers and other professionals to evaluate both the quality and abundance of cells for use in a wide range of applications. The collection of accurate counts for elements like cell culture nutrients, plasmid DNA, and transfection agents is a critical component for relevant analysis. The traditional cell counting method – manual assessment using hemocytometer and trypan blue – is tedious and time consuming. Additionally, sources of error arise in various areas, such as distortion of cell definition by human perception, and errors in the preparation and load of cell sample into the hemocytometer.

Recent advances in technology, low-cost imaging sensors, and embedded computing platforms allow for the development of accessible, portable, and low-cost microscopes. Specifically, a 3D-printed, low-cost optical microscope can perform cell imaging and reveal intricate cell details by adding a basic camera and Raspberry Pi for image acquisition and lighting control.

Furthermore, Artificial intelligence (AI) and machine learning (ML) techniques are significant and expanding fields, with the potential to automate the process of cell counting. Integrating Cell Counting Machine Learning Operations (CC-MLOps) is the key to accelerating operational improvements, minimizing errors, and enhancing model reliability. An example of one successful application of CC-MLOps is

automated cell counting of neuronal cells in rodent brain tissues by implementing GitHub integration, Neptune.ai experiment tracking, and Google Cloud Platform’s Vertex AI cloud deployment.

Unfortunately, amazing advancements like this require sophisticated hardware and software solutions – beyond the reach of most educators and researchers. This project aims to fill that gap by providing an automated cell counting solution with the open-source *Cellular Imaging Studio* program that works with the low-cost OpenFlexure 3D-printed microscope.

II. OBJECTIVES

A. Initial vs. Final Project Objectives

This project aimed to develop an accessible, low-cost solution for automated microscopy by integrating machine-learning-driven analysis with the OpenFlexure ecosystem. The project was executed in two phases. Phase 1 focused on hardware assembly and baseline software compatibility: assembling a 3D-printed OpenFlexure microscope [1], and ensuring compatibility with the existing OpenFlexure web interface. Objectives for Fall Quarter included developing the 3D-printed microscope and ensuring full functionality and compatibility with OpenFlexure’s Web Application Interface [2]. Additionally, we aimed to create a foundation for winter quarter by laying out the groundwork for the project’s machine learning, computer vision, and front-end-user experience components. [2]. Phase 2 concentrated on software and backend development: creating an open-source application capable of automated cell counting, motion-tracking, time-lapse imaging, and data export.



Fig. 1. OpenFlexure microscope build from Phase 1.

We successfully achieved all our primary objectives. The microscope was fully assembled and calibrated, shown in Figure 1, operating reliably with both the OpenFlexure interface and our custom application.

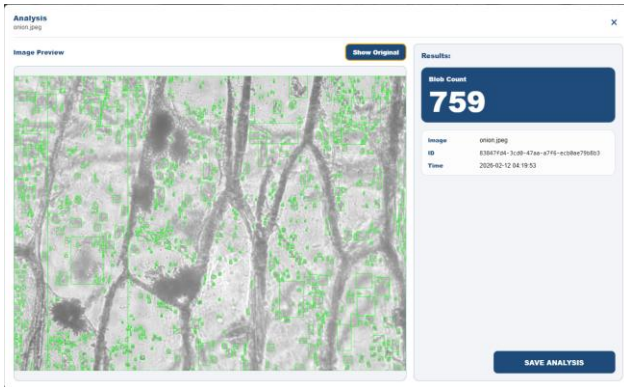


Fig. 2. View of automated cell counting results on custom user interface, Cellular Imaging Studio.

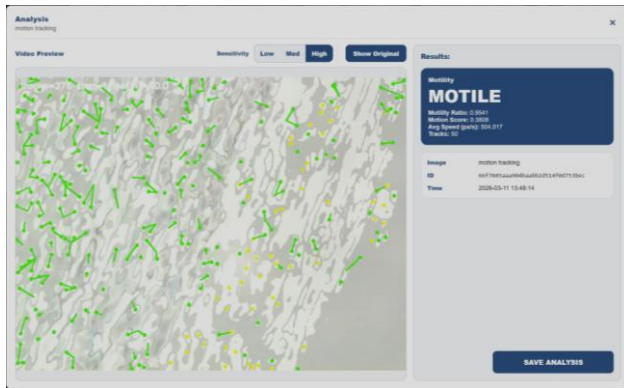


Fig. 3. View of motion tracking results on custom user interface, Cellular Imaging Studio

The machine learning pipelining was successfully integrated, enabling real-time cell counting, shown in Figure 2, and motion tracking from live microscope feeds, in Figure 3. The resulting custom application allows users to capture images, log CSV data, and perform time-lapse imaging, delivering an accessible tool for automated cell counting and motion tracking.

B. Challenges and Limitations

Despite successful implementation, two key limitations were encountered. First, the model accuracy proved highly sensitive to sample quality. Limited access to diverse samples and contract-enhancing dyes meant the models perform optimally under specific preparation conditions. Second, the limited volume of training data constrained model generalization. Generating a large, annotated set of data samples under varied conditions was not feasible within project timelines. Consequently, while the system successfully demonstrates automated insight generation, absolute accuracy does not yet match models trained on extensive public datasets.

III. PROJECT SETUP

A. Software

Our software is implemented with mix of OpenFlexure API and Python v3.11.9 with its subsequent libraries. Our Phase 1 (basic microscope design) software uses the flashed full desktop Raspbian-OpenFlexure OS and OpenFlexure Connect software which programs our Raspberry Pi v4 Model B to control the camera and motors of the microscope.

Phase 2 involves multiple Python libraries to develop a downloadable application that adds various functions to the base Openflexure software. The App UI is developed using the JavaScript framework Vue v3.5.26 for building pages and buttons, CSS for styling and layout, and NPM v11.6.2 and Node.js to build the ECMAScript/JavaScript core language, API and libraries for an installer. Vite is used to ensure fast hot-reload development.

Python libraries FastAPI, requests, and JSON are used to communicate with the Raspberry PI API and connect with Vue to pass information in JSON files. Pydantic is used to format requests in JSON files. Uvicorn [standard] is an ASGI server that runs the backend port. CSV, pathlib, and FFmpeg are used for downloading files to the user and path manipulation.

The OpenCV-Python (PyPI) segmentation model is used for AI development with help from NumPy to preprocess images for analysis.

B. Hardware

Our hardware design consists of a 3D-printed OpenFlexure high-resolution microscope [1], including a Raspberry Pi 4B to control the system, a Raspberry Pi Camera Board v2 for image capture, a BXEN-50S-11M-3C LED board for sample illumination, three 288YJ-48 5V stepper motors to shift the sample slide along three axes, and a Sangaboard v5 to control the LED board and three motors. We also added a 2.7 mm achromatic doublet and 40X plan objective lenses for image magnification and clarity.

IV. STANDARDS USED AND CONSIDERED

Our project incorporates several hardware, optical, and software standards to facilitate hardware operations, perform computer vision tasks, ensure accurate communication between subsystems, and support ML analyses.

A. Communication Standards

We implemented the IEEE 802.3 standard for data transfer between the microscope and client PC via Ethernet connection because it offers superior speed and data integrity, compared to wireless protocols.

We used RESTful API standards to maintain easy consistency and function call communication between the Raspberry Pi server and Client server and port connectivity. HTTP (RFC 9110) protocols and JSON (RFC 8259 / ECMA-404) file standards were also used to create consistent and reliable data retrieval and manipulation across the servers.

B. Actuator Control Standards

To control the LED and three stepper motors, we leveraged GPIO, I2C, and SPI protocols because of native support for them from the Raspberry Pi and all actuator units.

C. User Interface Standards

The development of our Cellular Imaging Studio app adheres to the W3C standards of HTML5 for page structure, CSS specifications for layout and styling, and the ECMAScript/JavaScript (ECMA-262) standard for interactive elements – via the Vue.js framework.

D. Camera Interface Standard

Image capture follows the MIPI Alliance CSI-2 camera interface standard, used by the Raspberry Pi Camera v2. This protocol supports high-speed, low latency image transfer directly to the Raspberry Pi. This allows for more stable performance than USB camera alternatives as well as enabling reliable autofocus and real-time ML analysis.

E. Objective Lens Standard

The microscope's optical system follows the DIN objective lens standard. The standard defines the focal length, tube length, and mechanical threading used in educational and research microscopes. Our 40x DIN compliant lens provides sufficient resolution for observing individual cells. This allows for accurate measurements during use.

F. Computer Vision Standards

We selected OpenCV for its robust library of computer vision tools and object recognition algorithms, paired with NumPy to manage image data in tensor format. Combined, these libraries provide an efficient solution for preprocessing captured images and supporting our ML methods for quantifying objects and tracking their movement.

G. Considered Standards

We evaluated several alternative standards during development. For imaging, we considered USB-based camera interfaces used by other OpenFlexure-supported cameras (Logitech C270, Arducam B0196) but determined the CSI-2 standard was a better fit. For computer vision, we considered Hogg's AI protocol for evaluating AI in clinical settings but rejected it due to poor performance on complex biological shapes – selecting the OpenCV standard instead. We also considered using TensorFlow to manage image data before determining it to be beyond the scope of this project and selecting NumPy instead.

V. SECURITY CONSIDERATIONS

A. Network Access Control

The first security issue we considered was network access to the microscope. Since the device can be run through a web interface or application through API calls, unauthorized users could potentially access the system if it is connected to an insecure network. Implementing a form of connection and secure communication protocols between apps and microscopes would help prevent unauthorized access.

B. Software Vulnerabilities

The custom software we created could create vulnerabilities in our system. Poorly validated inputs or insecure code could allow for command injection or unintended system access. Following secure coding practices and validating input helps reduce these risks.

C. AI Model Integrity

Because the system includes AI-based cell analysis, protecting the machine's learning model is important. If the training data or model is unexpectedly changed, it could produce incorrect results and give an inaccurate analysis. Verifying our model and protecting the training data can help maintain accuracy with our cell analysis.

D. Physical Hardware Tampering

Since the microscope uses 3D-printed components and embedded hardware, physical access could allow someone to tamper with the device. Someone can remove or unplug storage devices or modify internal connections. Restricting physical access to the pi and cables by using secure enclosures can reduce this risk.

E. Data Security

In the future, there are a few considerations that we must consider and implement into our microscope. The microscope captures images and experimental data that may be sensitive. If the device is compromised, someone could download, modify, or delete this data. Access control and encrypted storage can help protect research data.

F. Secure Software Updates

Lastly, if the microscope were developed into a product making our application downloadable to the public, secure updates would be necessary. Without protection, attackers could install malicious firmware or software updates. Using version verification and safe updates would help ensure only trusted software is installed.

VI. PROTOTYPES

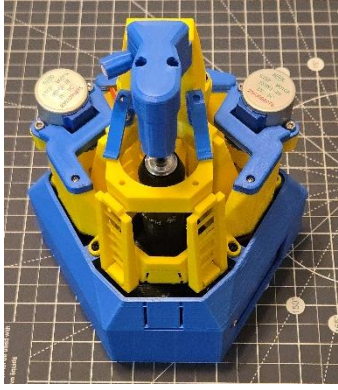


Fig. 4. OpenFlexure microscope build from Phase 1.

The sole physical prototype of our microscope was an implementation of the OpenFlexure high-resolution motorized microscope in [1] – shown in Figure 4 – serving as both our Fall 2025 quarter product goal and the foundation for our Winter 2026 final product goal. We validated the performance of our prototype by capturing an image from a sample of red onion cells in water – in addition to a successful battery of hardware and software diagnostics.

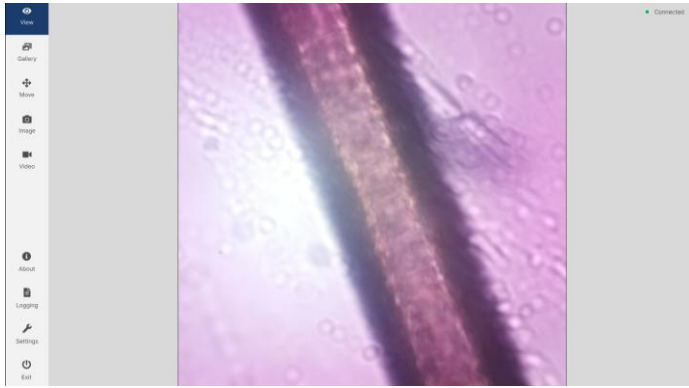


Fig. 5. GUI Prototype from Phase 1.

We had many iterations of our UI design as we fixed bugs and added features, resulting in the design shown in Figure 5. We modeled it after the *OpenFlexure Connect* app design with the addition of separate pages for image and video capturing and additional capabilities within the Gallery module.

VII. EXPERIMENTS AND RESULTS

A. Experimental Evaluation

To evaluate the functionality and performance of the developed microscope system, a series of experiments were conducted to test the hardware platform, imaging capabilities, software control interface, and machine learning analysis features. The experiments focused on verifying that the integrated system could reliably capture microscope images and videos, perform automated analysis, and present results through the custom desktop interface.

We conducted testing with several biological and environmental samples, including onion epidermal cells,

mosquito specimens, and water samples containing microscopic particles. These samples were selected because they provide a variety of visual structures and motion patterns that allow evaluation of both static image analysis and dynamic motion tracking algorithms.

The experiments evaluated these aspects of the system:

- 1) Mechanical stage control and positioning accuracy
- 2) Image and video captured through the OpenFlexure microscope
- 3) Functionality of the desktop control interface and data management features
- 4) Performance of the automated analysis algorithms for object detection and motion tracking

B. Mechanical System Performance

The microscope's motorized stage was tested using our custom backend route to OpenFlexure's URL request. To ensure reliable control over the X, Y, and Z axes, the 28BYJ-48 stepper motors were tested for smooth and repeatable motion. Users can perform movements through the UI control panel, allowing precise positioning of samples under the microscope objective.

The stage responded smoothly to movement commands issued from the user allowing for prompt feedback. The stepper motors allowed precise adjustments to focus and positioning, which enabled users to consistently bring microscopic structures into view.

Repeated movement across the sample area demonstrated stable mechanical behavior without drifting or unintended motion. The stage maintained its position when idle and allowed accurate repositioning of previously observed regions within the sample using our coordinate system. The developed coordinate system also allows easy stage centering with the press of a button, enabling the user to always orient themselves.

These results confirmed that the mechanical subsystem successfully integrates with the software interface and provides reliable positioning capabilities for microscopy experiments.

C. Imaging Performance

Image capture experiments were conducted using biological samples placed on microscope slides. The system successfully captured high-resolution images that clearly revealed microscopic structures.

Onion epidermal cells were used to test static imaging capabilities because their rectangular cell walls provide well-defined structures that are easy to identify. The captured images showed clear cell boundaries and consistent illumination across the field of view.

Mosquito samples were also imaged to test the system's ability to capture more complex biological structures. The images revealed fine details such as segmented body structures and appendages, demonstrating that the optical system can resolve small anatomical features.

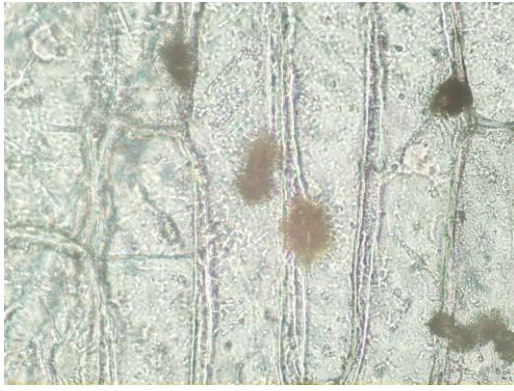


Fig. 6. Example microscope image of onion epidermal cells captured using the system.

Captured images, like the onion cells in Figure 6, were automatically stored and displayed within the application gallery, where users can view thumbnails, inspect full-resolution images, and access metadata for each capture.

D. Video Capture and Motion Analysis

In addition to static images, the system supports video recording to analyze dynamic samples. Video capture experiments were conducted using water samples containing microscopic particles and biological material.

These videos were processed using a motion tracking analysis pipeline implemented within the application. The algorithm identifies moving objects within the video frames and tracks their movement across time.

The analysis generates several quantitative measurements:

- 1) Motion score: Overall activity in the video
- 2) Motility ratio: Proportion of detected objects that exhibit movement
- 3) Average speed of tracked objects in pixels per second
- 4) Track count: Independent object trajectories detected

The software also produces visual overlays showing the paths of tracked objects across the video frames. These overlays allow users to visually verify that the algorithm is correctly identifying and following moving particles.

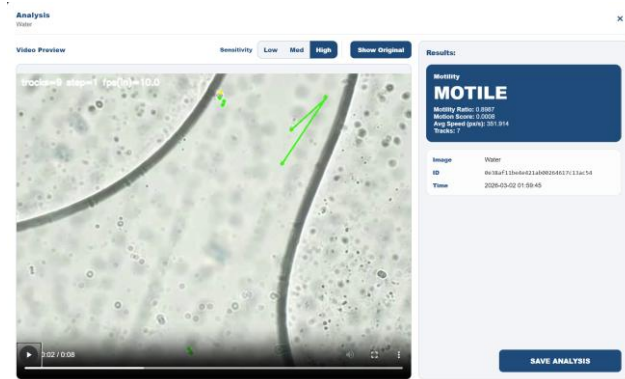


Fig. 7. Motion tracking overlay showing detected particle trajectories within a microscope video.

The movement tracking system, shown in Figure 7, successfully identified moving particles within water samples and produced consistent trajectory overlays, indicating that the analysis pipeline can extract meaningful motion information from microscope videos.

E. Automated Counting

The system includes an image analysis feature designed to automatically detect and count visible cellular structures in microscope images. This functionality uses an image segmentation and blob detection approach to identify distinct objects within a captured image.

When an image is analyzed, the algorithm processes the image to isolate potential cell regions and then counts the detected objects. A series of image processing occurs on the backend implementing OpenCV and NumPy. The results are displayed directly within the application and can be stored with the image metadata.

The analysis process also generates an overlay image highlighting the detected cells. This allows users to visually confirm that the algorithm's detections correspond to actual biological structures.

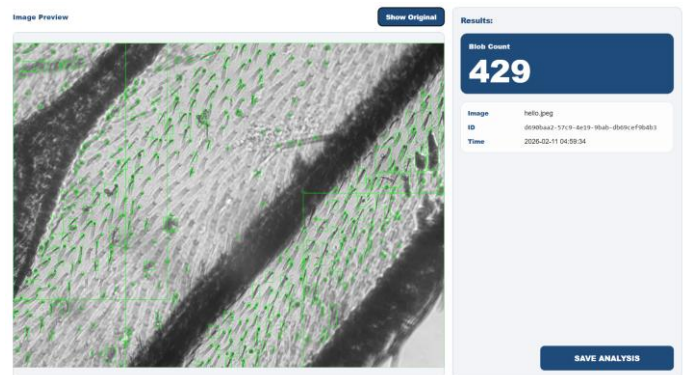


Fig. 8. Cell counting overlay showing detected wing hair follicles by the analysis algorithm.

Testing with a biological sample, like the mosquito cells in Figure 8, proved the algorithm was able to detect individual regions and produce accurate visual overlays, demonstrating the feasibility of automated cell counting within the platform.

F. Timelapse Imaging

A timelapse imaging feature was implemented to allow automated capture of a sequence of images over time. This feature is useful for observing gradual biological processes or monitoring changes in samples over extended periods.

The timelapse function repeatedly captures images at user-defined intervals and stores them in the gallery system. During testing, the feature successfully produced sequences of images that documented changes in sample positioning and object movement.

These sequences can later be reviewed within the gallery interface or exported for further analysis.

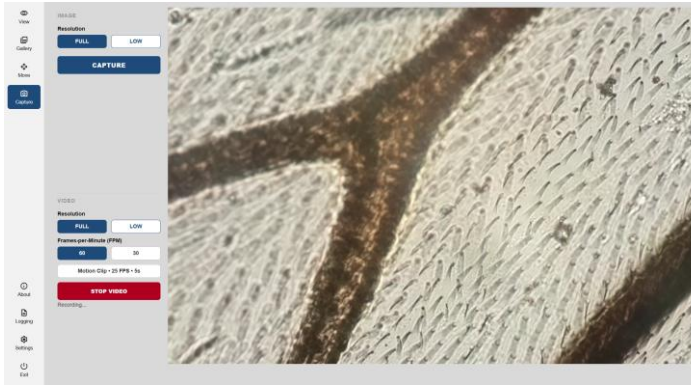


Fig. 9. Example from a timelapse capture demonstrating the recording interface.

The timelapse capability, shown in Figure 9, demonstrates that the system can perform extended automated observation tasks without continuous user interaction.

G. Software Interface and Data Management

The *Cellular Imaging Studio* application serves as the primary interface for interacting with the microscope system. Testing confirmed that the interface successfully integrates the major system components, including camera streaming, stage control, capturing tools, and analysis features.

The application provides several key functionalities:

- 1) Live camera preview
- 2) Manual stage movement controls
- 3) Image and video capture
- 4) Timelapse capture sequences
- 5) Automated image analysis
- 6) Motion tracking analysis for videos
- 7) Metadata editing for notes, annotations, and tags
- 8) Exporting analyses and media as CSV and ZIP files

Captured media is organized within a gallery view that allows users to browse images and videos, inspect metadata, and launch analysis tools directly from the interface. Additional features such as annotation editing, tag management, and file export allow users to manage microscopy data efficiently. The interface also supports light and dark display modes and includes an option to disable the live camera stream when preview is not required. These results demonstrate that the software interface provides a complete workflow for microscope operation, from data acquisition to analysis and export.

H. Summary of Results

The experiments demonstrate that the developed system successfully integrates mechanical control, optical imaging, and machine learning analysis within a single platform. The hardware subsystem provides stable stage movement and consistent image capture, while the custom software interface enables intuitive control of microscope functions. The analysis tools implemented in the *Cellular Imaging Studio* app can

detect cellular structures in static images and tracking motion within video samples. Together, these results show that the system can perform automated microscopy tasks such as cell detection, motion tracking, and timelapse imaging while maintaining an accessible user interface for data collection and analysis.

REFERENCES

- [1] R. Bowman and J. Stirling, "Assembly Instructions," The OpenFlexure Community, [Online]. Available: <https://build.openflexure.org/openflexure-microscope/v7.0.0-beta4/#high-resolution-motorised-microscope>. [Accessed 2 December 2025].
- [2] B. O. I. Group, "Web application interface," OpenFlexure Microscope Software, 2018. [Online]. Available: <https://openflexure-microscope-software.readthedocs.io/en/latest/webapp/index.html>. [Accessed 3 December 2025].
- [3] C. Richards, S. Sarkar, J. Kandell, R. Snyder and U. Lakshminpathy, "Assessing the suitability of cell counting methods during different stages of a cell processing workflow using an ISO 20391-2 guided study design and analysis," *Frontiers in Bioengineering and Biotechnology*, vol. 11, p. 1223227, August 2023.
- [4] ChemoMetec, "Manual vs. automated cell counting - the complete guide," 14 October 2024. [Online]. Available: <https://chemometec.com/manual-vs-automated-cell-counting/>. [Accessed 3 December 2025].
- [5] J. Christopher, R. McHugh, R. Craig, A. Roe, R. Bauer, B. Patton, G. McConnell and L. Rooney, "A 3D-printed optical microscope for low-cost histological imaging," *Journal of Microscopy*, vol. 298, pp. 274-282, June 2025.
- [6] M. Testi, L. Clissa, M. Ballabio, S. Ricciardi, F. Baldo, E. Frontoni, S. Moccia and G. Vessio, "Enhancing cell counting through MLOps: a structured approach for automated cell analysis," April 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2504.20126>. [Accessed 3 December 2025].